

Angular 12 For .Net Developers From Scratch

...



Mohd **Manzoor** Ahmed

(MCT | Founder Of www.ManzoorTheTrainer.com)

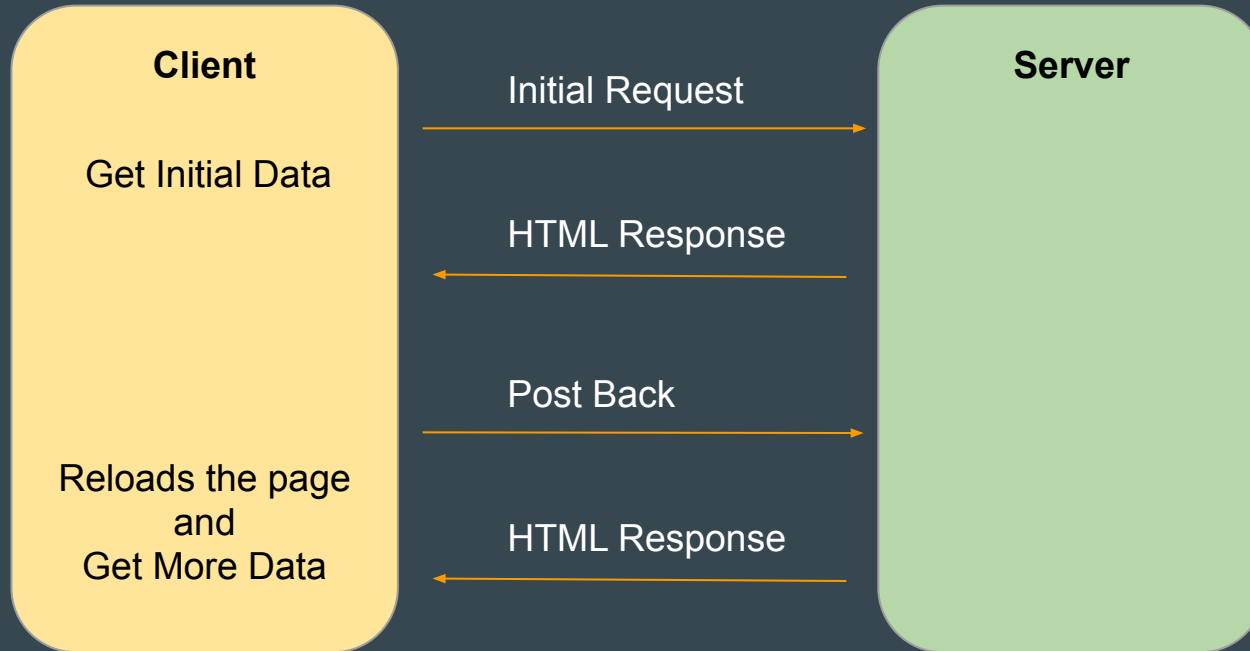
Prerequisite

- You should have some basic understanding of
 - HTML And CSS
 - JavaScript
 - TypeScript
 - Web APIs
- Or you can learn it here www.ManzoorTheTrainer.com

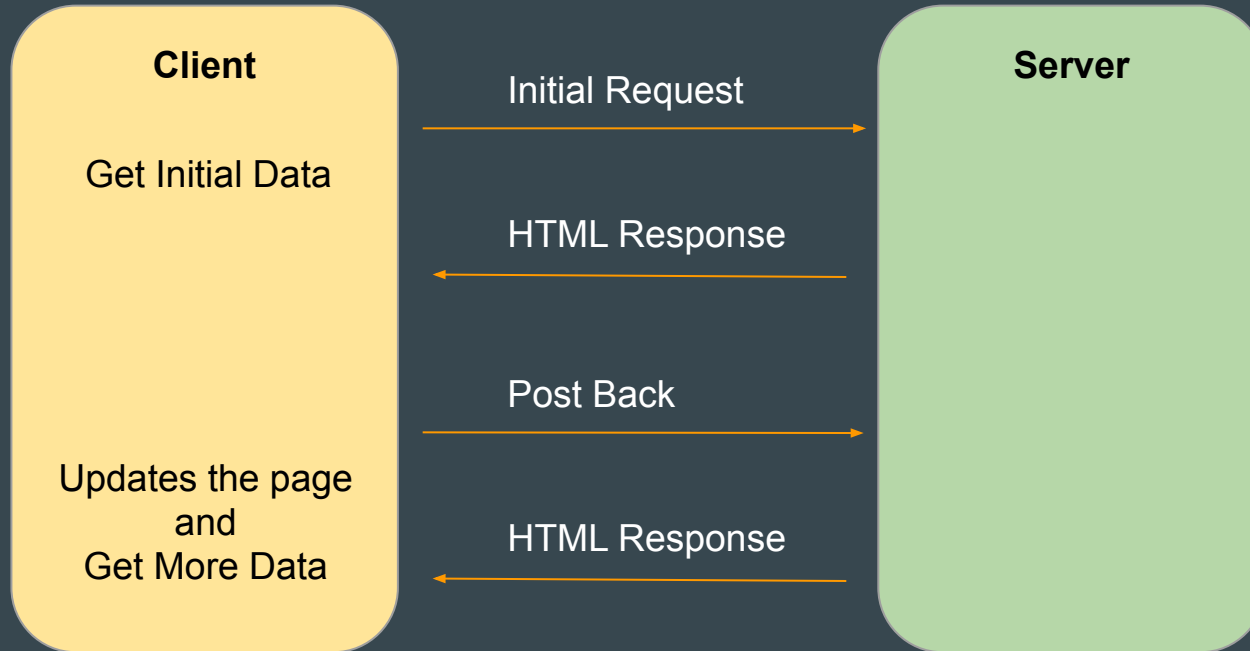
What is ASP.Net SPA?

- Get all necessary code – HTML, JavaScript, and CSS on the initial page load.
- Download appropriate features dynamically from server behind the scenes on response to a user action.
- Without reloading the complete page.
- One of the best examples of SPA is Gmail.

Page Life Cycle - Normal?



Page Life Cycle SPA?



How To Achieve SPA?

- It can be achieved with help of Web browser JavaScript frameworks, such as **Angular 2+**, AngularJS, knockout.js, Ember.js, ExtJS, React, etc

What is Angular 12?

- **Angular** is a typescript based open-source web application framework mainly maintained by Google.
- Its aims is to simplify both the development and the testing of client-side model–view–controller (**MVC**) and model–view–viewmodel (**MVVM**) architectures. (**MV***).
- The final version of Angular 2 was released on September 14, 2016.
- AngularJS version 2, 4, 5, 6, 7, 8, 9 ... are called as **Angular**

AngularJS 1.x Vs Angular 2+

AngularJS 1.x

- Controller Centric
- Uses JavaScript
- <https://manzoorthetrainer.com/p/angularjs-for-net-developers-from-scratch>

Angular 2, 4, 5, 7, 8, 9 ...

- Component Centric
- Uses TypeScript

Software And Tools Required

- Visual Studio Community Edition

<https://www.visualstudio.com/thank-you-downloading-visual-studio/?sku=Community&rel=15>

- NodeJs

<https://nodejs.org/en/download/>

Getting Started With **Angular 12**

Angular CLI : Command Line Interface tool to initialize, develop, scaffold and maintain Angular applications.

1. Start an **empty ASPNet Core** web application and **enable it for static files**
2. Open VS2019 command prompt in **admin mode** and Go to projects folder
3. Run this command -> **npm install @angular/cli --global** (Angular CLI Tool)
4. Run this command -> **ng new {app-name-here} --skip-install** (To create project)
5. Move content of **app folder** to **project root folder**
6. Open **angular.json** file and set "outputPath": "wwwroot"
7. Run this command -> **npm install** (Install All the Packages)(Longer Time Step)
8. Run this command -> **ng build** (To build the project)
9. Run this command -> **ng serve** (To run the project and get dynamic updates)

Component - Angular 12

Component :

- It's an element.
- It is created as .ts file
- `app.component.ts` is the root component.
- Add it to index.

← → ↻ ⓘ localhost:4200/index.html

Welcome - Angular UI

app.component.ts

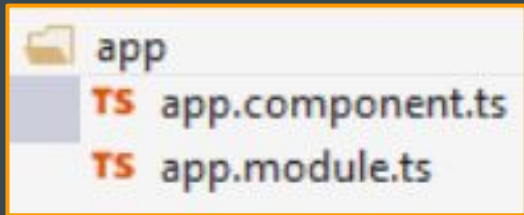
```
import { Component } from '@angular/core';  
  
@Component({  
  selector: 'app-root',  
  template: '<h1>Welcome - Angular UI </h1>',  
})  
  
export class AppComponent {  
}
```

index.html

```
<!doctype html>  
<html lang="en">  
<head>  
</head>  
<body>  
  <app-root></app-root>  
</body>  
</html>
```

app.module.ts

- **NgModule**: Describes how to combine and compile all components together.
- **BrowserModule**: provides services that are essential to launch and run a browser app.



app.module: An Angular main module class that describes how the application parts fit together.

app.module.ts

```
import { BrowserModule } from '@angular/platform-browser';
import { NgModule } from '@angular/core';
```

```
import { AppComponent } from './app.component';
```

```
@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule],
  bootstrap: [AppComponent]
})
```

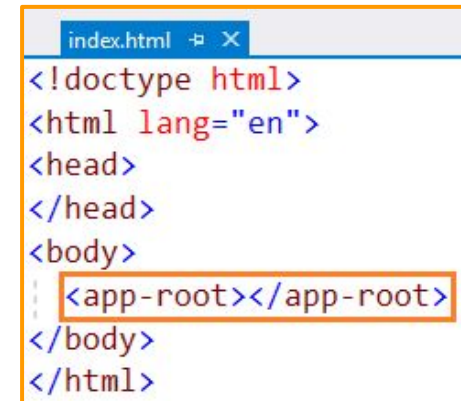
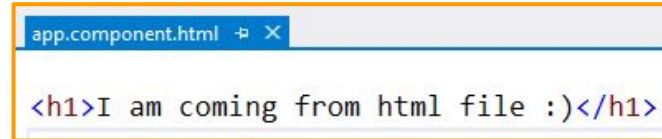
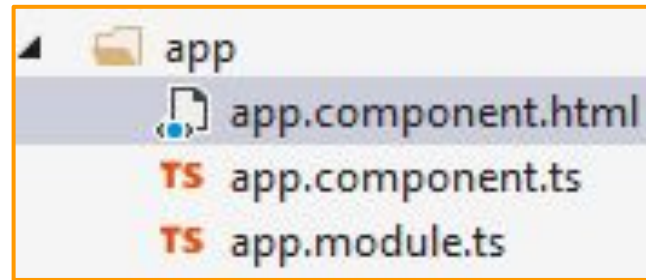
```
export class AppModule { }
```

Component's templateUrl

- **Template:** It's an HTML file rendering the component.
- **templateUrl:** It is the location of that html file

← → ↻ ⓘ localhost:4200/index.html

I am coming from html file :)



Understanding Basic Terminologies

- **Angular CLI** : Command Line Interface tool to initialize, develop, scaffold and maintain Angular applications
- **Components** : Basic building block of an UI in an Angular. It's an element.
- **app.module** : An Angular main module class describes how the application parts fit together.
 - **BrowserModule** : Provides services that are essential to launch and run a browser app.
 - **NgModule** : Describes how to compile a component's template.
 - declarations: [AppComponent]
 - imports: [BrowserModule]
 - bootstrap: [AppComponent]

Data Binding : Template Syntax

- Binds text content to an interpolated string. {{{}}

Needs **FormsModule** in imports:

- import { FormsModule } from '@angular/forms';
- Binds property to the result of expression. [ngModel]
- Two-way data binding. [(ngModel)]

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html'
})

export class AppComponent {
  name: string = "Manzoor The Trainer";
}
```

```
<h1 class="text-center">Welcome - Angular UI</h1>
```

Binds text content to an interpolated string

```
<h1> Welcome to {{{name}}} </h1>
```

Binds property to the result of expression

```
<input [ngModel]='name' type='text' />
```

Two-way data binding.

```
<input [(ngModel)]='name' type='text' />
```

Method Calling : (click) Event

- Calls method sayHello() when a click event is triggered on button element.

```
import { Component } from '@angular/core';  
@Component({  
  selector: 'app-root',  
  templateUrl: './app.component.html'  
})  
  
export class AppComponent {  
  name: string = "Manzoor The Trainer";  
  sayHello() {  
    alert("Hello");  
  }  
}
```

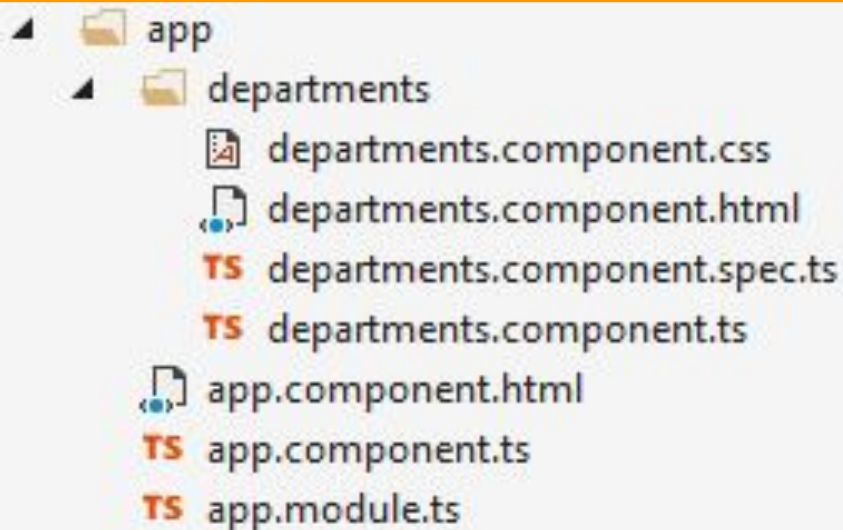
```
<h1>Welcome - Angular UI</h1>
```

```
<input (click)='sayHello()' type='button' value='ClickMe'>
```

Adding A New Component

- `ng g c {name-of-component}`

```
C:\Bootcamp\Angular8\FAUI\FAUI>ng g c departments
CREATE src/app/departments/departments.component.html (30 bytes)
CREATE src/app/departments/departments.component.spec.ts (663 bytes)
CREATE src/app/departments/departments.component.ts (289 bytes)
CREATE src/app/departments/departments.component.css (0 bytes)
UPDATE src/app/app.module.ts (391 bytes)
```



```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-departments',
  templateUrl: './departments.component.html',
  styleUrls: ['./departments.component.css']
})
export class DepartmentsComponent implements OnInit {

  constructor() { }

  ngOnInit() {
  }

}
```

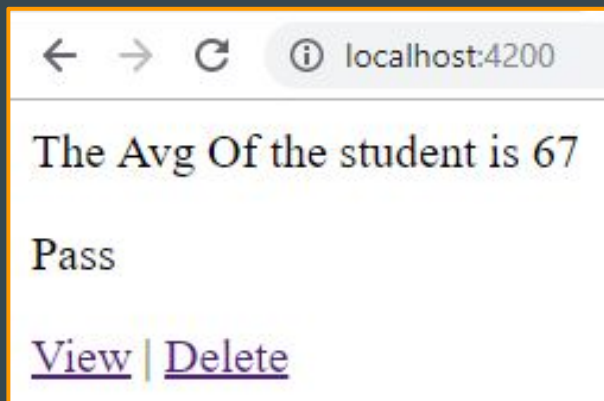
```
import { BrowserModule } from '@angular/platform-browser';
import { NgModule } from '@angular/core';

import { AppComponent } from './app.component';
import { DepartmentsComponent } from './departments/departments.component';

@NgModule({
  declarations: [AppComponent, DepartmentsComponent],
  imports: [BrowserModule],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Displaying Data With Directives (*ngIf)

- Single Variable
 - *ngIf



```
export class DepartmentsComponent implements OnInit {  
  avg: number = 67;  
  
  role: string = "Admin";  
  
  constructor() { }  
  
  ngOnInit() {  
  }  
}
```

```
<div>  
  The Avg Of the student is {{avg}}  
  <p *ngIf="avg>35">Pass</p>  
  <p *ngIf="avg<35">Fail</p>  
</div>  
  
<div>  
  <a href="#">View</a> |  
  <a href="#" *ngIf="role=='Admin'">Delete</a>  
</div>
```

Displaying Data With Directives (*ngFor)

- Array Of Variables
 - *ngFor

```
export class DepartmentsComponent implements OnInit {  
  
  contacts: string[] = ["9878765654", "9878765654", "9234323678"];  
  
  constructor() { }  
  
  ngOnInit() {  
  }  
}
```

```
<ul *ngFor="let contact of contacts">  
  <li>{{contact}}</li>  
</ul>
```

← → ↻ ⓘ localhost:4200

- 9878765654
- 9878765654
- 9234323678

Displaying JSON Data

- Single Object

```
dept = {  
  "did": 100,  
  "dName": "DevOps",  
  "description": "Its Dev Ops team"  
};
```

```
export class DepartmentsComponent implements OnInit {  
  
  dept = {  
    "did": 100,  
    "dName": "DevOps",  
    "description": "Its Dev Ops team"  
  };  
}
```

```
<div>  
  Did:{{dept.did}} <br/>  
  DName:{{dept.dName}} <br />  
  Description:{{dept.description}}  
</div>
```

← → ↻ ⓘ localhost:4200

Did:100
DName:DevOps
Description:Its Dev Ops team

Displaying JSON Data

- Array of Objects

```
depts = [{  
  "did": 100,  
  "dName": "DevOps",  
  "description": "Its Dev Ops team"  
},{  
  "did": 111,  
  "dName": "Testing",  
  "description": "Its Testing team"  
}];
```

```
export class DepartmentsComponent implements OnInit {  
  
  depts = [{ "did": 100, "dName": "DevOps", "description": "Its Dev Ops team" },  
            { "did": 111, "dName": "Testing", "description": "Its Testing team" }];  
  
}
```

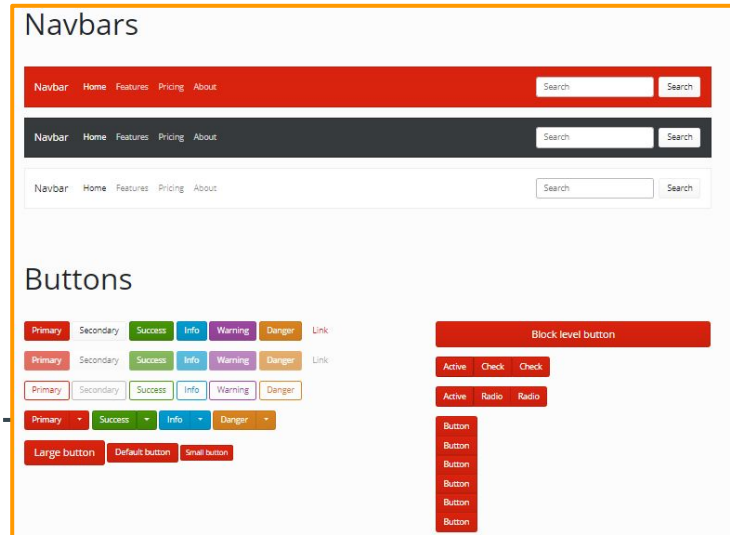
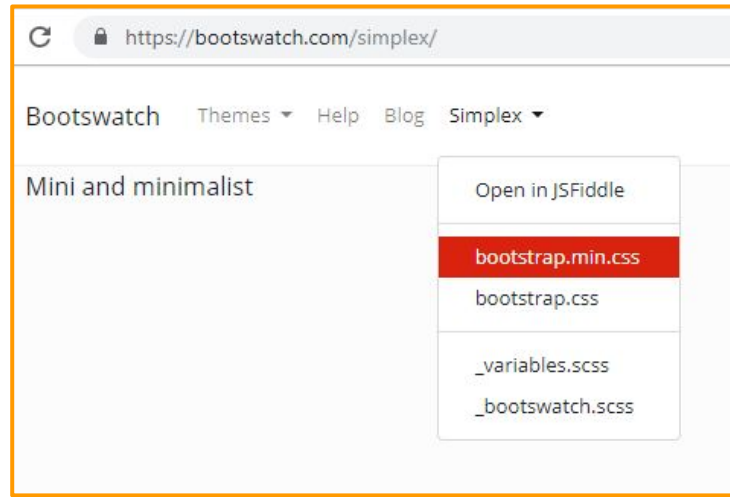
```
<div *ngFor="let dept of depts">  
  <div>  
    Did:{{dept.did}} <br />  
    DName:{{dept.dName}} <br />  
    Description:{{dept.description}}  
    <hr />  
  </div>  
</div>
```

← → ↻ ⓘ localhost:4200

Did:100	
DName:DevOps	
Description:Its Dev Ops team	
Did:111	
DName:Testing	
Description:Its Testing team	

Bootstrap 4

- In simple words Bootstrap is combination of predefined design classes in CSS and JavaScript file for developing responsive, mobile-first websites.
- Bootstrap 4 is completely free.
- <https://bootswatch.com/simplex/>
- Download bootstrap.min.css



Bootstrap 4 in Angular 12

→ npm install bootstrap jquery

→ Edit angular.json

- `"/node_modules/bootstrap/dist/css/bootstrap.css"`
- `"/node_modules/jquery/dist/jquery.js",`
`"/node_modules/bootstrap/dist/js/bootstrap.js"`

```
angular.json  X
{
  "build": {
    "builder": "@angular-devkit/build-angular:browser",
    "options": {
      "outputPath": "wwwroot",
      "index": "src/index.html",
      "main": "src/main.ts",
      "polyfills": "src/polyfills.ts",
      "tsConfig": "tsconfig.app.json",
      "aot": false,
      "assets": [
        "src/favicon.ico",
        "src/assets"
      ],
      "styles": [
        "node_modules/bootstrap/dist/css/bootstrap.min.css"
      ],
      "scripts": [
        "node_modules/jquery/dist/jquery.min.js",
        "node_modules/bootstrap/dist/js/bootstrap.min.js"
      ]
    }
  },
}
```

Displaying Data In Grid

enter to search		
Did	DName	Description
1004	Quality Control	Control Team
1005	Admin	Administration
1006	P-D-G	Product Development Group
1007	R&D	Research
1008	Billing	Get The Bills
« Previous 1 2 3 4 Next »		

Displaying Data In Grid

- **Paging**

→ npm install ngx-pagination
→ import { **NgxPaginationModule** }
from 'ngx-pagination'; // **app.module.ts**

- **Searching**

→ npm install ng2-search-filter
→ import { **Ng2SearchPipeModule** }
from 'ng2-search-filter'; // **app.module.ts**

- **Sorting**

→ npm install ngx-order-pipe
→ import { **OrderModule** }
from 'ngx-order-pipe'; in **app.module.ts**

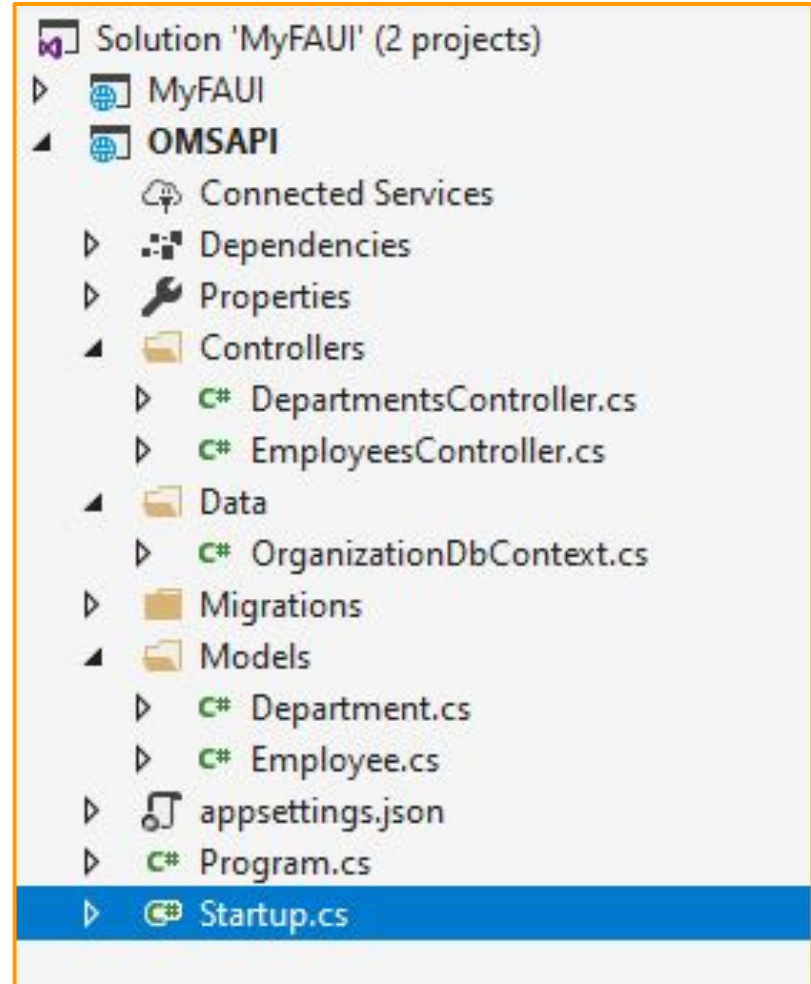
```
<div class="form-group">
  <input type="text" placeholder="enter to search"
    [(ngModel)]="search" class="form-control" />
</div>
<table class="table table-hover">
  <thead>
    <tr class="table-primary">
      <th (click)="sort('did')" style="cursor:pointer">Did</th>
      <th (click)="sort('dName')" style="cursor:pointer">DName</th>
      <th (click)="sort('description')" style="cursor:pointer">Description</th>
    </tr>
  </thead>
  <tbody>
    <tr *ngFor="let dept of depts| filter:search
      | paginate: { itemsPerPage: 5, currentPage: p }
      | orderBy:key: flag">
      <td>{{dept.did}}</td>
      <td>{{dept.dName}}</td>
      <td>{{dept.description}}</td>
    </tr>
  </tbody>
  <ul class="pagination">
    <pagination-controls (pageChange)="p = $event"></pagination-controls>
  </ul>
</table>
```

```
export class DepartmentsComponent implements OnInit {

  key: string;
  reverse: boolean = false;
  sort(val) {
    this.key = val;
    this.reverse = !this.reverse;
  }
}
```

Web API in .Net Core

1. Add .Net Core Web Application
2. Initial Setup
3. Add Models
4. Add DbContext
5. Generate Database
6. Insert Dummy Data
7. Add API Controllers
8. EnableCors
9. Update Port To 51294(Optional)



Call API Service - Angular 12

- HttpClientModule has the features to make HTTP request and responses in angular apps.

Get Method :

Step - 1:

→ In `app.module.ts`

→ import { `HttpClientModule` } from
'@angular/common/http';

Step - 2:

→ In `departments.component.ts`

→ import { `HttpClient` } from
'@angular/common/http';

```
<input type="button" (click)="getFromApi()"
      value="Get From Api" class="btn btn-primary" />
```

```
export class DepartmentsComponent implements OnInit {

  constructor(private http:HttpClient) {
  }
  depts = [];

  getFromApi() {
    this.http.get<any>("http://localhost:51294/api/Departments")
      .subscribe(res => {this.depts = res; });
  }
}
```

Get From Api		
enter to search		
Did	DName	Description
1008	Billing	Get The Bills
1007	R&D	Research
1006	P-D-G	Product Development Group
1005	Admin	Administration
1004	Quality Control	Control Team
« Previous 1 2 Next »		

Displaying Data In Grid With Dropdownlist Cascade

GetfromApi

Quality Control

enter to search

Eid

Name

Email

29

Channing Ford

aliquet.diam@ametfaucibusut.net

42

Lester Ewing

nec@Nullaegetmetus.org

48

Arden Mathis

Vestibulum.ut@tortorNunc.co.uk

49

John Berger

Proin@Donec.org

51

Keefe Hurley

Proin.nisl.sem@tacitisociosquad.ca

« Previous

1

2

3

Next »

Call API Service - Angular 10

Get Employee By DId Method :

Cascading dropdownlist with grid

```
[HttpGet("getEmployeesByDid/{did}")]
```

0 references | 0 requests | 0 exceptions

```
public async Task<IActionResult> GetEmployeesByDid(int did)
{
    return Ok(await _context.Employees
        .Where(x=>x.Did==did)
        .ToListAsync());
}
```

```
export class DepartmentsComponent implements OnInit {

    constructor(private http:HttpClient) {
    }
    apilink: string = "http://localhost:51294/api/";

    depts = [];
    getFromApi() {
        this.http.get<any>(this.apilink+"Departments")
            .subscribe(res => {this.depts = res; });
    }

    emps = [];
    did: number = 0;
    getEmpsByDidFromApi() {
        this.http.get<any>(this.apilink+"getEmployeesByDid/"+this.did)
            .subscribe(res => { this.emps = res; });
    }
}
```

```
<input type="button" (click)="getFromApi()"
    value="Get Departments From WebApi"
    class="btn btn-primary" />

<select [(ngModel)]="did" (change)="getEmpsByDidFromApi(did)">
    <option *ngFor="let dept of depts" value={{dept.did}}>
        {{dept.dName}}
    </option>
</select>

<table *ngIf="emps.length>0" class="table table-hover">
    <thead>...</thead>
    <tbody>...</tbody>
    <ul class="pagination">...</ul>
</table>
```

Template-driven forms - Majorly Used Form Elements

- import {FormsModule} from '@angular/forms' (app.module.ts)
 - TextBox
 - Button
 - Checkbox
 - RadioButton
 - DropDown List

Create Employee Form

Create Employee

☐ Male ☐ Female

Create

Insert Operation

html

```
<form>
  <fieldset>
    <legend>Create Employee</legend>
    <div class="form-group">
      <input type="text" class="form-control" name="name"
        [(ngModel)]="emp.empName" placeholder="Enter employee name">
    </div>
    <div class="form-group">
      <input type="email" class="form-control" name="email"
        [(ngModel)]="emp.email" placeholder="Enter employee email">
    </div>
    <div class="form-group">
      <input type="date" class="form-control" name="dob"
        [(ngModel)]="emp.dob" placeholder="Enter employee DOB">
    </div>
    <div class="form-group">
      <input type="radio" name="gender" [(ngModel)]="emp.gender" value="M"> Male
      <input type="radio" name="gender" [(ngModel)]="emp.gender" value="F"> Female
    </div>
    <div class="form-group">
      <select name="did" [(ngModel)]="emp.did" class="form-control">
        <option>Select A Department</option>
        <option *ngFor="let dept of depts" value="{{dept.did}}">
          {{dept.dName}}
        </option>
      </select>
    </div>
    <button type="submit" (click)="postEmployee()" class="btn btn-primary">Create</button>
  </fieldset>
</form>
```

component

```
export class EmployeesComponent implements OnInit {
  emp = {};
  depts = [];
  constructor(private http: HttpClient) { }

  ngOnInit() {
    this.http.get<any>("http://localhost:51294/api/Departments")
      .subscribe(res => { this.depts = res; });
  }

  postEmployee() {
    console.log(this.emp);
    this.http.post("http://localhost:51294/api/Employees", this.emp)
      .subscribe(res => {
        console.log(res);
        this.emp = {};
      }, err => {
        console.log(err);
      });
  }
}
```

Models In Angular10

- **Model** in Angular is a typescript class which represents the business object.
- Models make application strongly typed and gives readability to the application.

Eg: Employee as a model.
employee.ts

```
employee.ts → X
Angular: {} "employee"
export class Employee {
  empId: number;
  empName: string;
  email: string;
  gender: string;
  dob: Date;
  did: number;
  constructor() {
  }
}
```

```
emp: Employee = new Employee();
postEmployee() {
  console.log(this.emp);
  this.http.post<Employee>("http://localhost:51294/api/Employees", this.emp)
    .subscribe(res => {
      console.log(res);
      this.emp = new Employee();
    }, err => {
      console.log(err);
    });
}
```

Success Message & Server Side Errors

```
[HttpPost]
0 references | 0 requests | 0 exceptions
public async Task<IActionResult> PostEmployee(Employee employee)
{
    try
    {
        if (ModelState.IsValid)
        {
            _context.Employees.Add(employee);
            await _context.SaveChangesAsync();
            return CreatedAtAction("GetEmployee",
                new { id = employee.EmpId }, employee);
        }
        else
        {
            return BadRequest(ModelState);
        }
    }
    catch (Exception E)
    {
        var msg = (E.InnerException != null)
            ? (E.InnerException.Message) : (E.Message);
        return StatusCode(500, msg);
    }
}
```

- The INSERT statement conflicted with the FOREIGN KEY constraint "FK_Employee_Department_Did". The conflict occurred in database "Angular8OrgAPIDb", table "dbo.Department", column 'Did'. The statement has been terminated.

Create Employee

test

info@manzoorthetrainer.com

10/15/2019

☒ Male ☐ Female

Create

```
postEmployee() {
    console.log(this.emp);
    this.http.post<Employee>("http://localhost:51294/ap:
        .subscribe(res => {
            console.log(res);
            this.emp = new Employee();
            this.showSuccessMsg = true;
            this.showFailureMsg = false;
        }, err => {
            this.serverErrors = [];
            if (err.status === 400) {
                Object.keys(err.error).forEach(key => {
                    this.serverErrors.push(err.error[key][0]);
                });
            }
            else if (err.status === 500) {
                console.log(err);
                this.serverErrors.push(err.error);
            }
            this.showFailureMsg = true;
            this.showSuccessMsg = false;
        });
}
```

```
<div id="msg" *ngIf="showSuccessMsg" class="alert alert-success">
    <strong>Employee has been created successfully!</strong>
</div>
<ul *ngIf="showFailureMsg" class="alert alert-danger">
    <li *ngFor="let error of serverErrors">
        {{error}}
    </li>
</ul>
```

Client Validation - Template Referencing

- HTML5 attributes can be used for input fields validation in form.
 - Required: required
 - Required Email: type="email" required
 - Required Number With Range: type="number" min="" max=""
- State of the form and fields can be checked using these bool properties
 - untouched (Only for fields)
 - touched (Only for fields)
 - dirty (For both form and field)
 - invalid (For both form and field)
 - valid (For both form and field)
 - errors (For field)
 - submitted (Only for forms)

Client Validation

- Exporting ngModel to a local template variable.

Eg:

```
<form #empForm="ngForm">
  <input type="text" class="form-control"
    [(ngModel)]="emp.empName"
    #empNameControl="ngModel" name="name"
    placeholder="Enter employee name">
```

```
  <div *ngIf="(empNameControl.touched ||
    empForm.submitted) && empNameControl.invalid"
    class="alert alert-danger">
    Name is required!
  </div>
</form>
```

```
<form #empForm="ngForm" novalidate>
  <fieldset>
    <legend>Create Employee</legend>
    <div class="form-group">
      <input type="text" class="form-control" name="empName"
        #empNameControl="ngModel" required
        [(ngModel)]="emp.empName" placeholder="Enter employee name">
      <div *ngIf="(empNameControl.touched || empForm.submitted)
        && empNameControl.invalid" class="alert alert-danger">
        Name is required!
      </div>
    </div>
    <div class="form-group">...</div>
    <div class="form-group">...</div>
    <div class="form-group">...</div>
    <div class="form-group">...</div>
    <button type="submit" [disabled]="empForm.invalid"
      (click)="postEmployee(empForm)" class="btn btn-primary">Create</button>
  </fieldset>
</form>
```

Create Employee

Enter employee name

Name is required!

Enter employee email

Email is required!

mm/dd/yyyy

DOB is required!

☐ Male ☐ Female

Select A Department!

Create

Delete Operation

html

```
<table class="table table-hover">
  <thead>...</thead>
  <tbody>
    <tr *ngFor="let emp of emps | filter:search|orderBy:key:flag
      | paginate: { itemsPerPage: 5, currentPage: p }">
      <td>{{emp.empId}}</td>
      <td>{{emp.empName}}</td>
      <td>{{emp.email}}</td>
      <td>{{emp.gender}}</td>
      <td>{{emp.dob | date:'yyyy-MM-dd'}}</td>
      <td>
        <span (click)="getEmployees(emp.empId)"
          style="cursor:pointer;text-decoration:underline">Edit</span> |
        <span (click)="deleteEmployee(emp.empId)"
          style="cursor:pointer;text-decoration:underline">Delete</span>
      </td>
    </tr>
  </tbody>
</table>
<ul class="pagination">
  <pagination-controls (pageChange)="p = $event"></pagination-controls>
</ul>
```

component

```
deleteEmployee(empId: number) {
  if (confirm('are you sure?')) {
    this.http.delete<Employee>("http://localhost:51294/api/Employees/"+empId)
      .subscribe(res => {
        alert('Deleted Success');
      });
  }
}
```

Filters - Basic

Filters are special functions to format or transform the data using pipe character in an expression followed by a filter. `{{ model | filter }}`

- **lowercase** Format a string to lower case.
- **uppercase** Format a string to upper case.
- **currency** Format a number to a currency format.
- **number** Format a number to a string.
- **date** Format a date to a specified format.
- **json** Format an object to a JSON string.

Edit Operation

```
<table class="table table-hover">
  <thead>...</thead>
  <tbody>
    <tr *ngFor="let emp of emps | filter:search|orderBy:key:flag
      | paginate: { itemsPerPage: 5, currentPage: p }">
      <td>{{emp.empId}}</td>
      <td>{{emp.empName}}</td>
      <td>{{emp.email}}</td>
      <td>{{emp.gender}}</td>
      <td>{{emp.dob | date:'yyyy-MM-dd'}}</td>
      <td>
        <span (click)="getEmployees(emp.empId)"
          style="cursor:pointer;text-decoration:underline">Edit</span> |
        <span (click)="deleteEmployee(emp.empId)"
          style="cursor:pointer;text-decoration:underline">Delete</span>
      </td>
    </tr>
  </tbody>
</table>
<ul class="pagination">
  <pagination-controls (pageChange)="p = $event"></pagination-controls>
</ul>
```

```
<button *ngIf="!emp.empId" type="submit" [disabled]="empForm.invalid"
  (click)="postEmployee(empForm);empForm.reset()" class="btn btn-primary">
  Create
</button>
<button *ngIf="emp.empId" type="submit" [disabled]="empForm.invalid"
  (click)="putEmployee(empForm);empForm.reset()" class="btn btn-primary">
  Update
</button>
<button type="submit" *ngIf="emp.empId" [disabled]="empForm.invalid"
  (click)="emp={};" class="btn btn-primary">
  New
</button>
```

component

```
getEmployees(empId: number) {
  if (empId == 0) {
    this.http.get<Employee[]>("http://localhost:51294/api/Employees/")
      .subscribe(res => { this.emps = res; });
  }
  else {
    this.http.get<Employee>("http://localhost:51294/api/Employees/" + empId)
      .subscribe(res => { this.emp = res; });
  }
}
```

```
putEmployee(empForm) {
  if (!empForm.invalid) {
    console.log(this.emp);
    this.http.put<Employee>("http://localhost:51294/api/Employees/"+this.emp.empId, this.emp)
      .subscribe(res => {
        console.log(res);
        this.showSuccessMsg = true;
        this.showFailureMsg = false;
        this.getEmployees(0);
      }, err => {
        this.serverErrors = [];
        if (err.status === 400) {...}
        else if (err.status === 500) {...}
        this.showFailureMsg = true;
        this.showSuccessMsg = false;
      })
  }
}
```

Services In Angular10

- Components job is to present data to html.
- Services are also typescript files that handles data access job.
- It increase modularity and reusability.
- ng g s <serviceName>
- Add to the providers list in app.module
providers:[<serviceName>]

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Employee } from '../employees/employee';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class OrgService {

  constructor(private http: HttpClient) { }

  getEmployees(): Observable<Employee[]> {
    return this.http.get<Employee[]>("http://localhost:51294/api/Employees/");
  }
}
```

```
import { Component, OnInit } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Employee } from '../employee';
import { OrgService } from '../org.service';

@Component({
  selector: 'app-employees',
  templateUrl: './employees.component.html'
})
export class EmployeesComponent implements OnInit {

  constructor( private service: OrgService) { }

  getEmployees() {
    this.service.getEmployees().subscribe(res => { this.emps = res; });
  }
}
```

Routing In Angular10

1. `nav menu` in header
2. `<router-outlet>` in `app.component.html`
3. Routes in `app.module.ts`
4. `routerLink`

Web Storage Or State Management

- cookies
 - Values are included in every server request.
 - As any cookie, its size must be less than 4KB.
- sessionStorage
 - Data is lost when tab is closed
 - Data never transferred to the server
 - Storage limit is larger (5MB)
- localStorage
 - Persists when browser is closed.
 - Data stored locally without affecting website performance, more secure.
 - Data never transferred to the server.
 - Storage limit is larger (5MB).

Cookies - 6 Steps

1. npm install `ngx-cookie-service`
2. Add to app.module: import { `CookieService` } from 'ngx-cookie-service';
3. Add `CookieService` to module's providers.
4. Inject `CookieService` into the components constructor.
5. Set: `cookie.set("cookieName","cookieValue")`.
6. Get: `cookie.get("cookieName")`

sessionStorage - 6 Steps

1. npm install `angular-web-storage`
 2. Add to app.module: import { `AngularWebStorageModule` } from `'angular-web-storage'`;
 3. Add `AngularWebStorageModule` to module's imports.
 4. Inject `SessionStorageService` into the components constructor.
 5. Set: `session.set("sName","sValue")`.
 6. Get: `session.get("sName")`
-

localStorage - 6 Steps

1. npm install `angular-web-storage`
 2. Add to app.module: import {
`AngularWebStorageModule` } from
'angular-web-storage';
 3. Add `AngularWebStorageModule` to module's
imports.
 4. Inject `LocalStorageService` into the
components constructor.
 5. Set: `local.set("lsName","lsValue")`.
 6. Get: `local.get("lsName")`
-

Thanks